Diya Patel
24070126211

Assignment-6

diyaa-13/Java

```java
import java.time.LocalDate;

public class FullTimeEmployee extends Employee {
    protected String role;
    protected double baseSalary;
    protected double perfBonus;
    protected double hiringCommission;

    public FullTimeEmployee(String name, String pan, LocalDate joiningDate, String designation, Integer empID,
                            String role, double baseSalary, double perfBonus, double hiringCommission) {
        super(name, pan, joiningDate, designation, empID);
        this.role = role;
        this.baseSalary = baseSalary;
        this.perfBonus = perfBonus;
        this.hiringCommission = hiringCommission;
    }

    @Override
    public double calculateCTC() {
        if ("SWE".equalsIgnoreCase(role)) {
            return baseSalary + perfBonus;
        } else if ("HR".equalsIgnoreCase(role)) {
            return baseSalary + hiringCommission;
        } else {
            return baseSalary;
        }
    }
}
```

```java
import java.time.LocalDate;

public class InternEmployee extends Employee {
    private double stipend;
    private String mentorName;

    public InternEmployee(String name, String pan, LocalDate joiningDate, String designation, Integer empID,
                          double stipend, String mentorName) {
        super(name, pan, joiningDate, designation, empID);
        this.stipend = stipend;
        this.mentorName = mentorName;
    }

    @Override
    public double calculateCTC() {
        return stipend;
    }

    public String getMentorName() {
        return mentorName;
    }
}
```

```java
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        HashMap<Integer, Employee> employeeMap = new HashMap<>();

        employeeMap.put(key: 1, new FullTimeEmployee(name: "Alice", pan: "PAN123", LocalDate.of(year: 2020, month: 1, dayOfMonth: 15), designat… "Software Engineer", 101, "SWE", 60000, 5000, 2000));
        employeeMap.put(key: 2, new Manager(name: "Bob", pan: "PAN456", LocalDate.of(year: 2019, month: 3, dayOfMonth: 10), designat… "Project Manager", 102, "HR", 80000, 10000, 5000, 3000, 2000));
        employeeMap.put(key: 3, new ContractEmployee(name: "Charlie", pan: "PAN789", LocalDate.of(year: 2021, month: 6, dayOfMonth: 1), designat… "Contractor", 103, 160, 50));
        employeeMap.put(key: 4, new InternEmployee(name: "Diana", pan: "PAN234", LocalDate.of(year: 2024, month: 7, dayOfMonth: 1), designat… "Intern", 104, 18000, "Alice"));
        employeeMap.put(key: 5, new PartTimeEmployee(name: "Ethan", pan: "PAN567", LocalDate.of(year: 2022, month: 4, dayOfMonth: 5), designat… "Part-time Analyst", 105, 80, 40, 2500));
        employeeMap.put(key: 6, new SeniorManager(name: "Farah", pan: "PAN890", LocalDate.of(year: 2017, month: 11, dayOfMonth: 20), designat… "Senior Manager", 106,
                role: "HR", baseSalary: 110000, perfBonus: 15000, hiringCommission: 10000, … 6000, 5000, 12, 12000));

        System.out.println(x: "=== All Employees ===");
        printEmployees(new ArrayList<>(employeeMap.values()));

        System.out.println(x: "\n=== Managers (simple type check) ===");
        List<Employee> managers = new ArrayList<>();
        for (Employee employee : employeeMap.values()) {
            if (employee instanceof Manager) {
                managers.add(employee);
            }
        }
        printEmployees(managers);

        System.out.println(x: "\n=== CTC between 20000 and 90000 ===");
        List<Employee> ctcRange = new ArrayList<>();
        for (Employee employee : employeeMap.values()) {
            double ctc = employee.calculateCTC();
            if (ctc >= 20000 && ctc <= 90000) {
                ctcRange.add(employee);
            }
        }
        printEmployees(ctcRange);

        System.out.println(x: "\n=== Total CTC By Employee Type ===");
        Map<String, Double> typeSummary = new HashMap<>();
        for (Employee employee : employeeMap.values()) {
            String type = employee.getClass().getSimpleName();
            double current = typeSummary.getOrDefault(type, defaultValue: 0.0);
            typeSummary.put(type, current + employee.calculateCTC());
        }
        for (Map.Entry<String, Double> entry : typeSummary.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }
    }

    private static void printEmployees(List<Employee> employees) {
        for (Employee employee : employees) {
            System.out.println(
                    "Employee ID: " + employee.empID
                            + ", Name: " + employee.name
                            + ", Type: " + employee.getClass().getSimpleName()
                            + ", CTC: " + employee.calculateCTC());
        }
    }
}
```

```java
import java.time.LocalDate;

public class Manager extends FullTimeEmployee {
    private double ta;
    private double eduAllowance;

    public Manager(String name, String pan, LocalDate joiningDate, String designation, Integer empID,
                   String role, double baseSalary, double perfBonus, double hiringCommission,
                   double ta, double eduAllowance) {
        super(name, pan, joiningDate, designation, empID, role, baseSalary, perfBonus, hiringCommission);
        this.ta = ta;
        this.eduAllowance = eduAllowance;
    }

    @Override
    public double calculateCTC() {
        return super.calculateCTC() + ta + eduAllowance;
    }
}
```

```java
import java.time.LocalDate;

public class PartTimeEmployee extends ContractEmployee {
    private double fixedAllowance;

    public PartTimeEmployee(String name, String pan, LocalDate joiningDate, String designation, Integer empID,
                            int noOfHrs, double hourlyRate, double fixedAllowance) {
        super(name, pan, joiningDate, designation, empID, noOfHrs, hourlyRate);
        this.fixedAllowance = fixedAllowance;
    }

    @Override
    public double calculateCTC() {
        return super.calculateCTC() + fixedAllowance;
    }
}
```

```java
import java.time.LocalDate;

public class SeniorManager extends Manager {
    private int teamSize;
    private double leadershipAllowance;

    public SeniorManager(String name, String pan, LocalDate joiningDate, String designation, Integer empID,
                         String role, double baseSalary, double perfBonus, double hiringCommission,
                         double ta, double eduAllowance, int teamSize, double leadershipAllowance) {
        super(name, pan, joiningDate, designation, empID, role, baseSalary, perfBonus, hiringCommission, ta, eduAllowance
        this.teamSize = teamSize;
        this.leadershipAllowance = leadershipAllowance;
    }

    @Override
    public double calculateCTC() {
        return super.calculateCTC() + leadershipAllowance + (teamSize * 1000.0);
    }
}
```

Output:

```
PS C:\Users\vasum\Desktop\Diya\Java\Java>  c:; cd 'c:\Users\vasum\Desktop\Diya\Java\Java'; & 'C:\Desktop\Eclipse Adoptium\jdk-2
5.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vasum\AppData\Roami
ng\Code\User\workspaceStorage\11e68956c3244799e74c7a4ef6bd8268\redhat.java\jdt_ws\Java_efd641ee\bin' 'Main'
Employee ID: 102, Name: Bob, Type: Manager, CTC: 90000.0

=== Total CTC By Employee Type ===
FullTimeEmployee => 65000.0
SeniorManager => 155000.0
ContractEmployee => 8000.0
PartTimeEmployee => 5700.0
Manager => 90000.0
InternEmployee => 18000.0
PS C:\Users\vasum\Desktop\Diya\Java\Java>
```